

Le stack MERN

Axel@pift3225

NodeJS

Node.js est une [plateforme logicielle libre](#) en [JavaScript](#), orientée vers les applications [réseau évènementielles](#) hautement [concurrentes](#) qui doivent pouvoir [monter en charge](#).

Elle utilise la [machine virtuelle V8](#), la bibliothèque [libuv](#) pour sa [boucle d'évènements](#), et implémente sous [licence MIT](#) les spécifications [CommonJS](#).

Parmi les modules natifs de Node.js, on retrouve [http](#) qui permet le développement de [serveur HTTP](#). Ce qui autorise, lors du déploiement de sites internet et d'applications web développés avec Node.js, de ne pas installer et utiliser des [serveurs webs](#) tels que [Nginx](#) ou [Apache](#).

Concrètement, Node.js est un environnement bas niveau permettant l'exécution de [JavaScript](#) côté serveur.

NodeJS fournit aussi un gestionnaire de paquet pour l'installation de logiciel supplémentaires : npm.

Express

Express.js est un [framework](#) pour construire des [applications web](#) basées sur [Node.js](#)². C'est de fait le framework standard pour le développement de serveur en [Node.js](#)³. L'auteur original, TJ Holowaychuk, le décrit comme un serveur inspiré de [Sinatra](#)⁴ dans le sens qu'il est relativement minimaliste tout en permettant d'étendre ses fonctionnalités via des plugins.

Express va gérer les routes, les connexions à la base de données, l'authentification,...

MongoDB

MongoDB est développé depuis 2007 par MongoDB. Cette entreprise travaillait alors sur un système de [Cloud computing](#), informatique à données largement réparties, similaire au service [Google App Engine](#) de [Google](#).

Il est depuis devenu un des SGBD les plus utilisés, notamment pour les sites web de [Craigslist](#), [eBay](#), [Foursquare](#), [SourceForge.net](#), [Viacom](#), [pagesjaunes](#) et le [New York Times](#)³.

MongoDB (de l'anglais *humongous* qui peut être traduit par « énorme ») est un [système de gestion de base de données orienté documents](#), [répartissable sur un nombre quelconque d'ordinateurs](#) et ne nécessitant pas de schéma prédéfini des données. Il est écrit en [C++](#). Le serveur et les outils sont distribués sous [licence SSPL](#), les pilotes sous [licence Apache](#) et la documentation sous [licence Creative Commons](#)². Il fait partie de la mouvance [NoSQL](#).

Les données ne sont pas stockées sous forme de tables ayant des relations mais sous forme de collections d'objets ce qui sera particulièrement pratique avec JS.

Serveur NodeJS

Après avoir installé NodeJS, dans un répertoire vide :

```
$ npm init
```

This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.

package name: (ninjanode)

version: (1.0.0)

description:

entry point: (index.js) server.js

Point d'entrée: fichier exécuté lorsque le serveur est lancé.

test command:

git repository:

keywords:

author:

license: (ISC)

About to write to /ninjaNode/package.json:

```
{
  "name": "ninjanode",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Cette commande crée le fichier package.json dans lequel
seront enregistrées automatiquement les dépendances.

Is this OK? (yes)

Serveur NodeJS

Fichier server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Le serveur fonctionne !');
});

server.listen(3000);
```

Importe le module http.

Crée un serveur web. La fonction `createServer` reçoit en paramètres, la requête http et la réponse http.

Démarre le serveur et écoute sur le port 3000 (si aucun port n'est défini dans les variables d'environnement).

Pourquoi le port 3000?

```
$ node server
```

Lance l'exécution. Un navigateur ou Postman permettent d'afficher le texte en se connectant à `http://localhost:3000`.

Problème : à chaque modification du serveur ou de l'application, il faudra relancer la commande `node server`.
L'installation de nodemon permet la prise en compte des modifications sans relancer la commande :

```
$ npm install -g nodemon
$ nodemon server
```


Serveur NodeJS

Le serveur envoie toujours la même réponse. Il faut créer une application et la faire s'exécuter par le serveur. L'application est gérée par express qu'il faut installer.

```
$ npm install express
```

Noter qu'une dépendance est créée dans package.json

Fichier App.js

```
const express = require('express'); Importation du module
```

```
const app = express();
```

```
app.use((req, res) => {  
  res.json({ message: 'Le serveur fonctionne !' });  
});
```

```
module.exports = app; Exportation de l'application
```

Middleware qui reçoit la requête et la réponse en paramètre.
Écrit du JSON dans la réponse.

Fichier server.js

```
const http = require('http');  
const app = require('./app'); Importation de l'application
```

```
app.set('port', 3000);  
const server = http.createServer(app); Lancement de l'application
```

```
server.listen(3000);
```

Le résultat est le même mais on peut maintenant faire évoluer l'application en créant des middleware.

Application express

```
const express = require('express');

const app = express();

app.use((req, res, next) => {
  console.log('Requête reçue !');
  next();
});

app.use((req, res, next) => {
  res.status(201);
  next();
});

app.use((req, res, next) => {
  res.json({ message: 'Le serveur fonctionne !' });
  next();
});

app.use((req, res, next) => {
  console.log('Tout va bien !');
});

module.exports = app;
```

Les middleware peuvent recevoir un 3ème paramètre `next` qui est le middleware suivant. Ils peuvent l'appeler avec `next()`. Ainsi la requête est traitée par le premier puis passer au suivant...

Application express

```
const express = require('express');

const app = express();

app.use('/product', (req, res, next) => {
  const products = [
    {
      "id":67,
      "name":"Solo Piano 2",
      "description":"Album de Chilly Gonzales",
      "price":"15",
      "category_id":2,
      "category_name":"Electronics"
    },
    {
      "id":48,
      "name":"Bristol Shoes",
      "description":"Awesome shoes.",
      "price":"1000",
      "category_id":5,
      "category_name":"Movies"
    },
  ];
  res.status(200).json(products);
  next();
});

app.use((req, res, next) => {
  console.log('Tout va bien !');
});

module.exports = app;
```

Une route est configurée pour le middleware et c'est cette route qui permettra d'accéder aux produits.

Ici les produits sont renvoyés sous la forme de JSON. Pas encore d'interrogation de la base de données.

Application express

```
const express = require('express');
```

```
const app = express();
```

```
app.use((req, res, next) => {  
  res.setHeader('Access-Control-Allow-Origin', '*');  
  res.setHeader('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content, Accept, Content-Type, Authorization');  
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, PATCH, OPTIONS');  
  next();  
});
```

```
app.use('/product', (req, res, next) => {  
  const products = [  
    {  
      "id":67,  
      "name":"Solo Piano 2",  
      "description":"Album de Chilly Gonzales",  
      "price":"15",  
      "category_id":2,  
      "category_name":"Electronics"  
    },  
    {  
      "id":48,  
      "name":"Bristol Shoes",  
      "description":"Awesome shoes.",  
      "price":"1000",  
      "category_id":5,  
      "category_name":"Movies"  
    },  
  ],  
  res.status(200).json(products);  
  next();  
});
```

```
app.use((req, res, next) => {  
  console.log('Tout va bien !');  
});
```

```
module.exports = app;
```

Ajout des en-têtes CORS. Toutes les origines sont acceptées. Ce sera surtout nécessaire lors de l'intégration du frontend.
On pourrait aussi limiter les réponses à une seule origine.

Application express

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json());
```

Permet d'extraire le corps de la requête POST.

```
app.use((req, res, next) => {  
  res.setHeader('Access-Control-Allow-Origin', '*');  
  res.setHeader('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content, Accept, Content-Type, Authorization');  
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, PATCH, OPTIONS');  
  next();  
});
```

```
app.post('/product', (req, res, next) => {  
  console.log(req.body);  
  res.status(201).json({  
    message: 'Objet créé !'  
  });  
});
```

```
app.get('/product', (req, res, next) => {  
  ...  
});
```

```
module.exports = app;
```

On précise les routes avec le type de la requête http.

Problème: les données ne sont pas persistantes.

MongoDB



Axel's Org - ...



Access Manager

Billing

ORGANIZATION

Projects

Alerts 0

Activity Feed

Settings

Integrations

Access Manager

Billing

Support

Live Migration

AXEL'S ORG - 2024-06-04 > PROJECTS

Create a Project

Name Your Project

Add Members

Name Your Project

Project names have to be unique within the organization (and other restrictions).

api_db

Add Tags (Optional)

Use tags to efficiently label and categorize your projects. A project can have a maximum of 50 tags. You can modify tags for the project later. [Learn more](#)

Key	Value	Actions
Select a key or enter your own	: Select a value or enter your own	
+ Add tag		
		0 TAGS

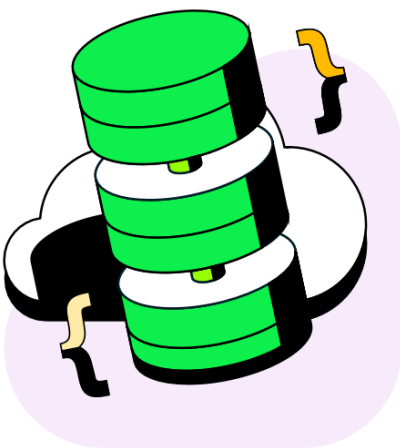
Cancel

Next

MongoDB.com permet la création gratuite d'un cluster mongoDB. Après avoir créé un projet, on peut créer un cluster.

MongoDB

MonProjet Overview



Create a cluster

Choose your cloud provider, region, and specs.

+ Create

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

M10

\$0.08/hour

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

Flex

From \$0.011/hour

Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU
5 GB	Shared	Shared

Free

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

✔ Free forever! Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name

You cannot change the name once the cluster is created.

Cluster0

Provider

aws

Google Cloud

Azure

Quick setup

☒ Automate security setup

☒ Preload sample dataset

Le cluster M0 sur AWS est gratuit.

MongoDB

Connect to Cluster0

1

2

3

Set up connection security

Choose a connection method

Connect

You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. [Read more](#)

- Add a connection IP address**

✓ Your current IP address (65.93.185.136) has been added to enable local connectivity. Add another later in [Network Access](#).
- Create a database user**

This first user will have [atlasAdmin](#) permissions for this project.

We autogenerated a username and password. You can use this or create your own.

i You'll need your database user's credentials in the next step. Copy the database user password.

Username

dbadmin

Password

.....

SHOW

Copy

Create Database User

Close

Choose a connection method

Add IP Access List Entry

Atlas only allows client connections to a cluster from entries in the project's IP Access List. Each entry should either be a single IP address or a CIDR-notated range of addresses. [Learn more](#).

ALLOW ACCESS FROM ANYWHERE

Access List Entry:

0.0.0.0/0

Comment:

Optional comment describing this entry

☐

This entry is temporary and will be deleted in

6 hours

Cancel

Confirm

Network Access : donner l'accès à tous.

MongoDB

Connect to Cluster0

✓

✓

3

Set up connection securityChoose a connection methodConnect

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver	Version
Node.js	5.5 or later

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

```
mongodb+srv://dbadmin:<password>@cluster0.rhggmbd.mongodb.net/?
retryWrites=true&w=majority&appName=Cluster0
```

Replace **<password>** with the password for the **dbadmin** user. Ensure any option params are [URL encoded](#).

RESOURCES

[Get started with the Node.js Driver](#)[Access your Database Users](#)

[Node.js Starter Sample App](#)[Troubleshoot Connections](#)

Go Back

Done

Dans les méthodes de connexion, on obtient la chaine de connexion à utiliser ainsi que les instructions pour installer mongodb dans l'application.

Application express

```
$ npm install mongoose
```

Fichier App.js

```
const express = require('express');  
const mongoose = require('mongoose');  
  
mongoose.connect('mongodb+srv://dbadmin:*****@cluster0.rhggmbd.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0')  
  .then(() => console.log('Connexion à MongoDB réussie !'))  
  .catch(() => console.log('Connexion à MongoDB échouée !'));
```

Importation du module mongoose qui permet de se connecter à MongoDB et configuration de la chaîne de connexion au sein d'une promesse.

```
Connexion à MongoDB réussie !  
[nodemon] restarting due to changes...  
[nodemon] starting `node server.js`  
Connexion à MongoDB réussie !  
└─
```

Dans la console

Création du schéma

Fichier produit.js

```
const mongoose = require('mongoose');
```

Importation du module mongoose.

```
const produit = mongoose.Schema({  
  name: { type: String, required: true },  
  description: { type: String, required: true },  
  category_id: { type: Number, required: true },  
  price: { type: Number, required: true },  
});
```

Création du schéma et définition des types des différentes données.

```
module.exports = mongoose.model('Produit', produit);
```

Exportation du schéma.

Création et lecture

```
const Produit = require('./produit');
```

```
app.post('/product', (req, res, next) => {  
  delete req.body._id;  
  const produit = new Produit({  
    ...req.body  
  });  
  produit.save()  
    .then(() => res.status(201).json({ message: 'Objet enregistré !'}))  
    .catch(error => res.status(400).json({ error }));  
});
```

S'assurer qu'un id n'existe pas déjà et laisser MongoDB le créer.

Créer chaque champs du produit automatiquement. Il est aussi possible d'utiliser `name = req.body.name ...`

Création de l'objet dans la BD à l'aide de `save()` au sein d'une promesse.

```
app.get('/product', (req, res, next) => {  
  Produit.find()  
    .then(produits => res.status(200).json(produits))  
    .catch(error => res.status(400).json({ error }));  
  next();  
});
```

L'affichage des produits se fait en lisant tous les produits grâce à `find()`.

Ajout des autres routes

```
app.post('/product', (req, res, next) => {  
  delete req.body._id;  
  const produit = new Produit({  
    ...req.body  
  });  
  produit.save()  
    .then(() => res.status(201).json({ message: 'Objet enregistré !'}))  
    .catch(error => res.status(400).json({ error }));  
});
```

```
app.get('/product', (req, res, next) => {  
  Produit.find()  
    .then(produits => res.status(200).json(produits))  
    .catch(error => res.status(400).json({ error }));  
  next();  
});
```

```
app.get('/product/:id', (req, res, next) => {  
  Produit.findOne({ _id: req.params.id })  
    .then(produit => res.status(200).json(produit))  
    .catch(error => res.status(404).json({ error }));  
});
```

Ajout de la route pour lire un produit en passant en paramètres l'id du produit.

La fonction findOne prenant en paramètre l'identifiant du produit au format JSON permet de récupérer un seul produit.

Ajout des autres routes

```
app.post('/product', (req, res, next) => {
  delete req.body._id;
  const produit = new Produit({
    ...req.body
  });
  produit.save()
    .then(() => res.status(201).json({ message: 'Objet enregistré !'}))
    .catch(error => res.status(400).json({ error }));
});
```

```
app.get('/product', (req, res, next) => {
  Produit.find()
    .then(produits => res.status(200).json(produits))
    .catch(error => res.status(400).json({ error }));
  next();
});
```

```
app.get('/product/:id', (req, res, next) => {
  Produit.findOne({ _id: req.params.id })
    .then(produit => res.status(200).json(produit))
    .catch(error => res.status(404).json({ error }));
});
```

```
app.put('/product/:id', (req, res, next) => {
  Produit.updateOne({ _id: req.params.id }, { ...req.body, _id: req.params.id })
    .then(() => res.status(200).json({ message: 'Objet modifié !'}))
    .catch(error => res.status(400).json({ error }));
});
```

```
app.delete('/product/:id', (req, res, next) => {
  Produit.deleteOne({ _id: req.params.id })
    .then(() => res.status(200).json({ message: 'Objet supprimé !'}))
    .catch(error => res.status(400).json({ error }));
});
```

Problème : la logique des routes et les fonctions métiers sont mélangées.

On veut l'id passé en paramètre dans l'URL, pas celui qui viendrait du corps de la requête.

Utilisation des fonctions
updateOne() et
deleteOne().

Optimisation

metier.js

```
const Produit = require('./produit');

exports.creerProduit = (req, res, next) => {
  const produit = new Produit({
    name: req.body.name,
    description: req.body.description,
    price: req.body.price,
    category_id: req.body.category_id
  });
  produit.save()
    .then(() => { res.status(201).json({ message: 'Produit créé!' });})
    .catch((error) => { res.status(400).json({ error: error }); });
};

exports.lireProduit = (req, res, next) => {
  Produit.findOne({
    _id: req.params.id
  })
    .then((produit) => { res.status(200).json(produit);})
    .catch((error) => { res.status(404).json({ error: error }); });
};

exports.modifierProduit = (req, res, next) => {
  const produit = new produit({
    _id: req.params.id,
    name: req.body.name,
    description: req.body.description,
    price: req.body.price,
    category_id: req.body.category_id
  });
  Produit.updateOne({_id: req.params.id}, produit)
    .then(() => { res.status(201).json({ message: 'Produit mis à jour!' });})
    .catch((error) => { res.status(400).json({ error: error }); });
};

exports.supprimerProduit = (req, res, next) => {
  Produit.deleteOne({_id: req.params.id})
    .then(() => { res.status(200).json({ message: 'Supprimé!' });})
    .catch((error) => { res.status(400).json({ error: error }); });
};

exports.lireTous = (req, res, next) => {
  Produit.find()
    .then((produits) => { res.status(200).json(produits);})
    .catch((error) => { res.status(400).json({ error: error }); });
};
```

Le but est de séparer la logique métier de la logique des routes.

App.js

```
const express = require('express');
const mongoose = require('mongoose');

const routesProduits = require('./routeur');

mongoose.connect('mongodb+srv://dbadmin:JLaygmgC42Yyp0bj@cluster0.rhggmbd.mongodb.net/?retryW
  .then(() => console.log('Connexion à MongoDB réussie !'))
  .catch(() => console.log('Connexion à MongoDB échouée !'));

const app = express();

app.use(express.json());

app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content, Accept,
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, PATCH, OPTIONS');
  next();
});

app.use('/product', routesProduits);

module.exports = app;
```

```
const express = require('express');
const router = express.Router();
const controleur = require('./metier');

router.get('/', controleur.lireTous);
router.post('/', controleur.creerProduit);
router.get('/:id', controleur.lireProduit);
router.put('/:id', controleur.modifierProduit);
router.delete('/:id', controleur.supprimerProduit);

module.exports = router;
```

Factorisation des routes.

routeur.js

Les routes appellent les fonctions du fichier metier.js.

Authentication

On veut pouvoir authentifier des utilisateurs en se basant sur une adresse courriel et un mot de passe.

Pour être certain que l'adresse courriel est unique :

```
$ npm install mongoose-unique-validator
```

Puis création d'un nouveau schéma pour l'utilisateur :

```
const mongoose = require('mongoose');
const uniqueValidator = require('mongoose-unique-validator');

const userSchema = mongoose.Schema({
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});

userSchema.plugin(uniqueValidator);

module.exports = mongoose.model('User', userSchema);
```


Authentication

Fichier userControler.js

```
exports.signup = (req, res, next) => {  
  };  
  
exports.login = (req, res, next) => {  
  };
```

Fichier userRoutes.js

```
const express = require('express');  
const router = express.Router();  
  
const userCtrl = require('./userControler');  
  
router.post('/enregistrement', userCtrl.signup);  
router.post('/connexion', userCtrl.login);  
  
module.exports = router;
```

Fichier App.js

```
const express = require('express');  
const mongoose = require('mongoose');  
  
const routesProduits = require('./routeur');  
const routesUser = require('./userRoutes');  
  
mongoose.connect('mongodb+srv://dbadmin:JLaygmgC42Yyp0bj@cluster0.rhggmbd.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0')  
  .then(() => console.log('Connexion à MongoDB réussie !'))  
  .catch(() => console.log('Connexion à MongoDB échouée !'));
```

Problème : le mot de passe ne doit pas être stocké en clair dans la BD.
Solution :
`$ npm install bcrypt`

Authentication

bcrypt permet de stocker le mot de passe de l'utilisateur en le cryptant de façon unidirectionnelle (on ne peut le décrypter). Lorsque l'utilisateur saisit son mot de passe pour authentification, il est à nouveau crypté et bcrypt compare les deux versions cryptées.

```
const User = require('./user');  
const bcrypt = require('bcrypt');  
const jwt = require('jsonwebtoken');
```

À placer en tête du fichier.

```
exports.signup = (req, res, next) => {  
  bcrypt.hash(req.body.password, 10)  
    .then(hash => {  
      const user = new User({  
        email: req.body.email,  
        password: hash  
      });  
      user.save()  
        .then(() => res.status(201).json({ message: 'Utilisateur créé !' })))  
        .catch(error => res.status(400).json({ error }));  
    })  
    .catch(error => res.status(500).json({ error }));  
};
```

Effectue 10 passages pour le hash. Plus il y a de passages, plus c'est sécuritaire mais plus c'est long.

Stocker l'utilisateur et son mdp dans la BD

```
exports.login = (req, res, next) => {  
  User.findOne({ email: req.body.email })  
    .then(user => {  
      if (!user) {  
        return res.status(401).json({ message: 'Paire login/mot de passe incorrecte' });  
      }  
      bcrypt.compare(req.body.password, user.password)  
        .then(valid => {  
          if (!valid) {  
            return res.status(401).json({ message: 'Paire login/mot de passe incorrecte' });  
          }  
          res.status(200).json({  
            userId: user._id,  
            token: 'TOKEN'  
          });  
        })  
        .catch(error => res.status(500).json({ error }));  
    })  
    .catch(error => res.status(500).json({ error }));  
};
```

Trouver l'utilisateur avec le bon login.

Crypter le mdp saisi et le comparer à celui stocké dans la BD

Renvoyer un TOKEN qui prouvera l'authentification (ici juste une chaîne de caractères).

Authentication

\$ npm install jsonwebtoken Installation de jsonwebtoken.

```
const jwt = require('jsonwebtoken');

exports.login = (req, res, next) => {
  User.findOne({ email: req.body.email })
    .then(user => {
      if (!user) {
        return res.status(401).json({ message: 'Paire login/mot de passe incorrecte' });
      }
      bcrypt.compare(req.body.password, user.password)
        .then(valid => {
          if (!valid) {
            return res.status(401).json({ message: 'Paire login/mot de passe incorrecte' });
          }
          res.status(200).json({
            userId: user._id,
            token: jwt.sign(
              {userId: user._id},
              'PHRASE_ALEATOIRE_TRES_LONGUE',
              { expiresIn: '24h' }
            )
          });
        })
        .catch(error => res.status(500).json({ error }));
    })
    .catch(error => res.status(500).json({ error }));
};
```

La fonction jwt.sign prend trois arguments :

- Le payload: ce qui doit être encrypté dans le token, ici le user id.
- Une phrase aléatoire permettant de crypter en toute sécurité.
- Une durée de vie du token (facultatif).

Authentication

Pour implémenter l'authentification sur nos routes précédentes :

```
const express = require('express');
const router = express.Router();
const controleur = require('./metier');
const auth = require('./userController');

router.get('/', auth, controleur.lireTous);
router.post('/', auth, controleur.creerProduit);
router.get('/:id', auth, controleur.lireProduit);
router.put('/:id', auth, controleur.modifierProduit);
router.delete('/:id', auth, controleur.supprimerProduit);

module.exports = router;
```

L'ordre est important, le contrôleur d'authentification doit être placé avant les autres contrôleurs.

Dépendances

L'application fonctionne. Beaucoup de paquets ont été installés.
On les retrouve tous dans le fichier `package.json`.

```
{
  "name": "ninjanode",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "bcrypt": "^5.1.1",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.5.0",
    "mongoose-unique-validator": "^5.0.1"
  }
}
```

Pour copier l'application, il suffit de fournir les fichiers JS et `package.json`.
Le répertoire `node_modules` n'a pas à être transféré.
Il suffit d'exécuter :
`$ npm install`
Toutes les dépendances seront alors installées.

Frontend : React

Read Products

+ Create Product

Name	Price	Category	Action
Solo Piano 2	18	2	Read Edit Delete
Bristol Shoes	999	5	Read Edit Delete
Nike Shoes for Men	12999	3	Read Edit Delete
Amanda Waller Shirt	333	1	Read Edit Delete
Wallet	799	6	Read Edit Delete
Another product	555	2	Read Edit Delete
Abercrombie Allen Brook Shirt	70	1	Read Edit Delete
Huawei Y300	100	2	Read Edit Delete
Sony Smart Watch	300	6	Read Edit Delete
Spalding Watch	199	1	Read Edit Delete
Samsung Galaxy Tab 10.1	259	2	Read Edit Delete
Lenovo Laptop	399	2	Read Edit Delete
LG P880 4X HD	336	3	Read Edit Delete

Création d’une interface permettant d’interagir avec l’API REST développée avec NodeJS.
L’interface doit permettre d’ajouter des produits, d’afficher les détails, de modifier, de créer et de supprimer des produits.

Frontend : React

Dans un répertoire séparé, créer un projet React :

```
$ npm create vite@latest
```

Choisir : React - JavaScript

```
$ cd vite-project
```

```
$ npm install
```

```
$ npm run dev
```

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMHjY6hW+ALEwIH" crossorigin="anonymous">
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDz0xhy9GkcIdslK1eN7N6jIeHz" crossorigin="anonymous"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/bootbox.js/6.0.0/bootbox.min.js" crossorigin="anonymous"></script>
  </body>
</body>
</html>
```

Dans le fichier index.html, on importe Bootstrap (CSS et JS).

Frontend : React

Fichier App.jsx

Reçoit en paramètres les gestionnaires d'évènements et les données à afficher.

```
import { useEffect, useState } from 'react'
```

```
function Product({ id, name, price, category_id, onReadClick, onEditClick, onDeleteClick }){
  return (
    <>
      <td>{name}</td>
      <td>{price}</td>
      <td>{category_id}</td>
      <td>
        <button className='btn btn-primary m-r-10px read-one-product-button' data-id={id} onClick={onReadClick}><span
        className='glyphicon glyphicon-eye-open'></span> Read</button>
        <span> </span>
        <button className='btn btn-info m-r-10px update-product-button' data-id={id} onClick={onEditClick}><span className='glyphicon
        glyphicon-edit' onClick={onEditClick}></span> Edit</button>
        <span> </span>
        <button className='btn btn-danger delete-product-button' data-id={id} onClick={onDeleteClick}><span className='glyphicon
        glyphicon-remove'></span> Delete</button>
      </td>
    </>
  )
}
```

Création d'un composant Product qui sera appelé lors de l'affichage de la liste des produits.

Name	Price	Category	Action
Solo Piano 2	18	2	Read Edit Delete
Bristol Shoes	999	5	Read Edit Delete
Nike Shoes for Men	12999	3	Read Edit Delete

Frontend : React

```
const [produits, setProduits] = useState(null);
const [read, setRead] = useState(null);
const [edit, setEdit] = useState(null);
const [del, setDel] = useState(null);
const [update, setUpdate] = useState(null);
const [create, setCreate] = useState(false);
const [addProduct, setAddProduct] = useState(false);
```

```
useEffect(() => {
  if(read){
    fetch('http://localhost:3000/product/' + read, { method: 'GET' })
      .then(rep => rep.json())
      .then( rep => {
        setProduits(rep);
      })
      .catch(err => console.log(err));
  }else if(del){
    fetch('http://localhost:3000/product/' + del, { method: 'DELETE' })
      .then(rep => rep.json())
      .then( rep => {
        setDel(null);
      })
      .catch(err => console.log(err));
  }else if(edit){
    fetch('http://localhost:3000/product/' + edit, { method: 'GET' })
      .then(rep => rep.json())
      .then( rep => {
        setProduits(rep);
      })
      .catch(err => console.log(err));
  }else if(update){
    fetch('http://localhost:3000/product/' + update,
    {
      method: 'PUT',
      headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(
        {
          "name": produits.name,
          "description": produits.description,
          "category_id": produits.category_id,
          "price": produits.price
        }
      )
    })
    .then(() => setUpdate(null));
  }
}, [read, edit, del, update, addProduct]);
```

Le composant principal
avec ses variables
d'état.

useEffect est utilisé
pour synchroniser le
composant à chaque
requête vers l'API REST
(synchronisation avec
une logique externe).
À noter: useEffect sera
appelé à chaque
modification d'une des
variables d'état
présente dans la liste
des dépendances.

```
}else if(addProduct){
  fetch('http://localhost:3000/product/',
  {
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(
      {
        "name": produits.name,
        "description": produits.description,
        "category_id": produits.category_id,
        "price": produits.price
      }
    )
  })
  .then(() => setAddProduct(false));
}else{
  fetch('http://localhost:3000/product', { method: 'GET' })
    .then(rep => rep.json())
    .then( rep => {
      const liste = rep.slice();
      setProduits(liste);
    })
    .catch(err => console.log(err));
}, [read, edit, del, update, addProduct]);
```


Frontend : Détails

```
if(read){
  return(
    <>
    <div className='container'>
      <div className='page-header'>
        <h1 id='page-title'>Product Details</h1>
        <div id='page-content'>
          <div id='read-products' className='btn btn-primary pull-right m-b-15px read-products-button' onClick={handleProducts}>
            <span className='glyphicon glyphicon-list'></span> Read Products
          </div>
          <table className='table table-bordered table-hover'>
            <tbody>
              <tr>
                <td>Name</td><td>{produits.name}</td>
              </tr>
              <tr>
                <td>Price</td><td>{produits.price}</td>
              </tr>
              <tr>
                <td>Description</td><td>{produits.description}</td>
              </tr>
              <tr>
                <td>Category</td><td>{produits.category_id}</td>
              </tr>
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </>
)
}
```

Bouton pour revenir à la liste des produits avec son gestionnaire d'évènements.

 Read Products

Lorsque l'on souhaite accéder au détail d'un produit.

Frontend : Formulaire

```
else if(edit || create){  
  return(  
    <>
```

Le même formulaire est utilisé pour la création et la modification.

```
    <div className='container'>  
      <div className='page-header'>  
        <h1 id='page-title'>{edit ? "Edit Product" : "Create Product"}</h1>  
        <div id='page-content'>  
          <div id='read-products' className='btn btn-primary pull-right m-b-15px read-products-button' onClick={handleProducts}>  
            <span className='glyphicon glyphicon-list'></span> Read Products  
          </div>
```

☰ Read Products

Le titre de la page change.

```
    <form method='post' action='#'>  
      <table className='table table-bordered table-hover'>  
        <tbody>  
          <tr>  
            <td>Name</td>  
            <td><input  
              type='text'  
              value={produits.name || ""}  
              onChange={(e) => {  
                setProduits(previousState => {  
                  return {...previousState, name: e.target.value}  
                } );  
              }}  
            /></td>  
          </tr>  
          <tr>  
            <td>Price</td>  
            <td><input  
              type='number'  
              value={produits.price || ""}  
              onChange={(e) => {  
                setProduits(previousState => {  
                  return {...previousState, price: e.target.value}  
                } );  
              }}  
            />  
          </td>  
        </tr>
```

Chaque champs DOIT être géré de cette façon :

- Initialisation à "" si le champs produit correspondant n'a pas de valeur.
- Récupération de tous les champs existant du produit et mise à jour uniquement du champs concerné.

Frontend : Formulaire

```
<tr>
  <td>Description</td>
  <td><input
    type='text'
    value={produits.description || ""}
    onChange={(e) => {
      setProduits(previousState => {
        return {...previousState, description: e.target.value}
      } );
    }}
  />
</td>
</tr>
```

```
<tr>
  <td>Category</td>
  <td><input
    type='number'
    value={produits.category_id || ""}
    onChange={(e) => {
      setProduits(previousState => {
        return {...previousState, category_id: e.target.value}
      } );
    }}
  />
</td>
</tr>
```

```
<tr>
  <td></td><td>
    <button type='submit' className='btn btn-info' onClick={handleSubmit}>
      <span className='glyphicon glyphicon-edit'></span>
      { edit ? "Update Product" : "Create Product" }
    </button>
  </td>
</tr>
</tbody>
</table>
</form>
</div>
</div>
</div>
</div>
```

Create Product

 Read Products

Name	
Price	
Description	
Category	
Create Product	

Bouton de soumission du formulaire.

Frontend : Liste des produits

La variable d'état est un tableau de produits.

```
else if(Array.isArray(products) && !read && !edit && !del){
  return (
    <>
    <div className='container'>
      <div className='page-header'>
        <h1 id='page-title'>Read Products</h1>
        <div id='page-content'>
          <button id='create-product' className='btn btn-primary pull-right m-b-15px create-product-button' onClick={handleCreate}>
            <span className='glyphicon glyphicon-plus'></span> Create Product
          </button>
          <table className='table table-bordered table-hover'>
            <thead>
              <tr>
                <th className='w-25-pct'>Name</th>
                <th className='w-10-pct'>Price</th>
                <th className='w-15-pct'>Category</th>
                <th className='w-25-pct text-align-center'>Action</th>
              </tr>
            </thead>
            <tbody>
              {products.map((produit, index) => (
                <tr key={index}>
                  <Product
                    id={produit._id}
                    name={produit.name}
                    price={produit.price}
                    category_id={produit.category_id}
                    onReadClick={() => handleReadClick(produit._id)}
                    onEditClick={() => handleEditClick(produit._id)}
                    onDeleteClick={() => handleDeleteClick(produit._id)}
                  />
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  )
}
```

Read Products

Name	Price	Category	Action
Solo Piano 2	18	2	Read Edit Delete

Utilisation de la fonction map pour parcourir le tableau de produits :

- Premier paramètre : l'élément du tableau
- Deuxième paramètre : l'indice de l'élément dans le tableau qui permet de créer la clef pour chacun des TR.

Insertion du composant Produit en lui passant les paramètres attendus.

Frontend : Évènements

```
function handleReadClick(i){
  setRead(i);
}

function handleEditClick(i){
  setEdit(i);
}

function handleDeleteClick(i){
  if(confirm("Are you sure?"))
    setDel(i);
}

function handleProducts(){
  if(read)
    setRead(null);
  else if(edit)
    setEdit(null);
  else if(create)
    setCreate(false);
}

function handleCreate(){
  setCreate(true);
}

function handleSubmit(){
  if(edit){
    setUpdate(edit);
    setEdit(null);
  }else{
    setCreate(false);
    setAddProduct(true);
  }
}
```

Les différents gestionnaires d'évènements modifient seulement les variables d'état dont dépend useEffect ce qui appelle useEffect à chaque fois et qui permet d'effectuer les tests présents dans useEffect.

Frontend : Fonctionnement 1

```
<button id='create-product' className='btn btn-primary pull-right m-b-15px create-product-button' onClick={handleCreate}>
  <span className='glyphicon glyphicon-plus'></span> Create Product
</button>
```

Pas une dépendance
de useEffect mais
nouveau rendu tout de
même.

```
function handleCreate(){
  setCreate(true);
}
```

Read Products

Name	Price	Category	Action
Solo Piano 2	18	2	<button>Read</button> <button>Edit</button> <button>Delete</button>

+ Create Product

Toutes les
variables sont à
leur état par défaut
: affichage de la
liste des produits.

Dépendance de
UseEffect.

```
function handleSubmit(){
  if(edit){
    setUpdate(edit);
    setEdit(null);
  }else{
    setCreate(false);
    setAddProduct(true);
  }
}
```

```
else if(addProduct){
  fetch('http://localhost:3000/product/',
  {
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(
      {
        "name": produits.name,
        "description": produits.description,
        "category_id": produits.category_id,
        "price": produits.price
      }
    )
  })
  .then(() => setAddProduct(false));
}
```

Create Product

Name	
Price	
Description	
Category	

Read Products Create Product

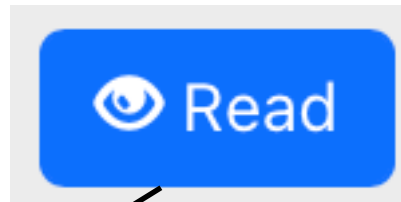
```
else if(edit || create){
  return(
    <>
      <div className='container'>
        <div className='page-header'>
          <h1 id='page-title'>{edit ? "Edit Product" : "Create Product"}</h1>
        <div id='page-content'>
          <div id='read-products' className='btn btn-primary pull-right m-b-15px read-products-button' onClick={handleProducts}>
            <span className='glyphicon glyphicon-list'></span> Read Products
          </div>
          <form method='post' action='#'>
            <table className='table table-bordered table-hover'>
              <tbody>
                <tr>
                  <td>Name</td>
                  <td><input
                    type='text'
                    value={produits.name || ""}
                    onChange={(e) => {
                      setProduits(previousState => {
                        return {...previousState, name: e.target.value}
                      } );
                    }}
                  ></td></tr>
                <tr>
                  <td></td><td>
                    <button type='submit' className='btn btn-info' onClick={handleSubmit}>
                      <span className='glyphicon glyphicon-edit'></span>
                      { edit ? "Update Product" : "Create Product" }
                    </button>
                  </td>
                </tr>
              </tbody>
            </table>
          </form>
        </div>
      </div>
    </>
  )
}
```


Frontend : Fonctionnement 2

```
function Product({ id, name, price, category_id, onReadClick, onEditClick, onDeleteClick }) {
  return (
    <>
      <td>{name}</td>
      <td>{price}</td>
      <td>{category_id}</td>
      <td>
        <button className='btn btn-primary m-r-10px read-one-product-button' data-id={id} onClick={onReadClick}><span className='glyphicon glyphicon-eye-open'></span> Read</button>
        <span> </span>
        <button className='btn btn-info m-r-10px update-product-button' data-id={id} onClick={onEditClick}><span className='glyphicon glyphicon-edit' onClick={onEditClick}></span> Edit</button>
        <span> </span>
        <button className='btn btn-danger delete-product-button' data-id={id} onClick={onDeleteClick}><span className='glyphicon glyphicon-remove'></span> Delete</button>
      </td>
    </>
  )
}
```

```
<tbody>
  {produits.map((produit, index) => (
    <tr key={index}>
      <Product
        id={produit._id}
        name={produit.name}
        price={produit.price}
        category_id={produit.category_id}
        onReadClick={() => handleReadClick(produit._id)}
        onEditClick={() => handleEditClick(produit._id)}
        onDeleteClick={() => handleDeleteClick(produit._id)}
      />
    </tr>
  ))}
</tbody>
```

```
function handleReadClick(i){
  setRead(i);
}
```



```
if(read){
  fetch('http://localhost:3000/product/' + read, { method: 'GET' })
    .then(rep => rep.json())
    .then( rep => {
      setProduits(rep);
    })
    .catch(err => console.log(err));
}
```

```
if(read){
  return(
    <>
      <div className='container'>
        <div className='page-header'>
          <h1 id='page-title'>Product Details </h1>
          <div id='page-content'>
            <div id='read-products' className='btn btn-primary pull-right m-b-15px rea
              <span className='glyphicon glyphicon-list'></span> Read Products
            </div>
            <table className='table table-bordered table-hover'>
              <tbody>
                <tr>
                  <td>Name</td><td>{produits.name}</td>
                </tr>
                <tr>
                  <td>Price</td><td>{produits.price}</td>
                </tr>
              </tbody>
            </table>
          </div>
        </div>
      </>
    )
  }
```


Frontend : Fonctionnement 3

```
if(read){
  return(
    <>
    <div className='container'>
      <div className='page-header'>
        <h1 id='page-title'>Product Details </h1>
        <div id='page-content'>
          <div id='read-products' className='btn btn-primary pull-right m-b-15px read-products-button' onClick={handleProducts}>
            <span className='glyphicon glyphicon-list'></span> Read Products
          </div>
          <table className='table table-bordered table-hover'>
            <tbody>
              <tr>
                <td>Name</td><td>{produits.name}</td>
              </tr>
              <tr>
                <td>Price</td><td>{produits.price}</td>
              </tr>
            </tbody>
          </table>
        </div>
      </div>
    </div>
  )
}
```

Product Details

Read Products

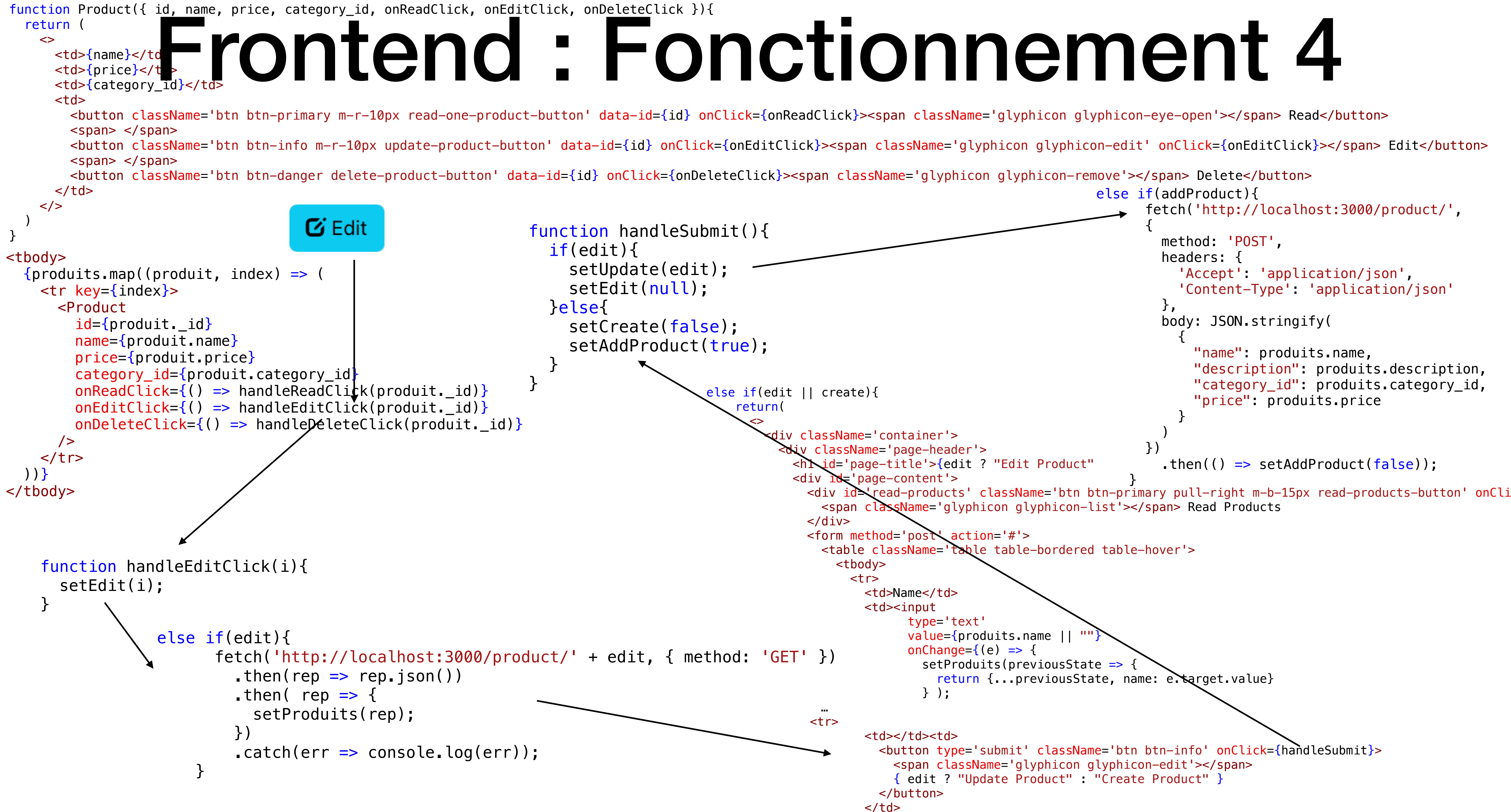
Name	Solo Piano 2
Price	18
Description	Album de Chilly Gonzales
Category	2

```
function handleProducts(){
  if(read)
    setRead(null);
  else if(edit)
    setEdit(null);
  else if(create)
    setCreate(false);
}
```

Toutes les
variables sont à
leur état par défaut
: affichage de la
liste des produits.

```
else{
  fetch('http://localhost:3000/product', { method: 'GET' })
    .then(rep => rep.json())
    .then( rep => {
      const liste = rep.slice();
      setProduits(liste);
    })
    .catch(err => console.log(err));
}
```

Frontend : Fonctionnement 4




```
function Product({ id, name, price, category_id, onReadClick, onEditClick, onDeleteClick }){
  return (
    <>
      <td>{name}</td>
      <td>{price}</td>
      <td>{category_id}</td>
      <td>
        <button className='btn btn-primary m-r-10px read-one-product-button' data-id={id} onClick={onReadClick}><span className='glyphicon glyphicon-eye-open'></span> Read</button>
        <span> </span>
        <button className='btn btn-info m-r-10px update-product-button' data-id={id} onClick={onEditClick}><span className='glyphicon glyphicon-edit'></span> Edit</button>
        <span> </span>
        <button className='btn btn-danger delete-product-button' data-id={id} onClick={onDeleteClick}><span className='glyphicon glyphicon-remove'></span> Delete</button>
      </td>
    </>
  )
}
```

```
<tbody>
  {produits.map((produit, index) => (
    <tr key={index}>
      <Product
        id={produit._id}
        name={produit.name}
        price={produit.price}
        category_id={produit.category_id}
        onReadClick={() => handleReadClick(produit._id)}
        onEditClick={() => handleEditClick(produit._id)}
        onDeleteClick={() => handleDeleteClick(produit._id)}
      />
    </tr>
  ))}
</tbody>
```



```
function handleDeleteClick(i){
  if(confirm("Are you sure?"))
    setDel(i);
}
```

```
else if(del){
  fetch('http://localhost:3000/product/' + del, { method: 'DELETE' })
    .then(rep => rep.json())
    .then( rep => {
      setDel(null);
    })
    .catch(err => console.log(err));
}
```

Toutes les variables sont à leur état par défaut : affichage de la liste des produits.

Déploiement

```
{
  "name": "ninjafrontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint . --ext js,jsx --report-unused-disable-directives --max-warnings 0",
    "preview": "vite preview",
    "server": "vite --host 0.0.0.0 --port 80"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.3.3",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "eslint": "^8.57.0",
    "eslint-plugin-react": "^7.34.2",
    "eslint-plugin-react-hooks": "^4.6.2",
    "eslint-plugin-react-refresh": "^0.4.7",
    "vite": "^5.3.1"
  }
}
```

```
$ sudo npm run server
```

Le frontend tourne maintenant sur le port 80.

Authentication (serveur)

user.js

```
const mongoose = require('mongoose');
const uniqueValidator = require('mongoose-unique-validator');

const userSchema = mongoose.Schema({
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});

userSchema.plugin(uniqueValidator);

module.exports = mongoose.model('User', userSchema);
```

Création de la collection d'utilisateur sur MongoDB.

userRoutes.js

```
const express = require('express');
const router = express.Router();

const userCtrl = require('./userController');

router.post('/enregistrement', userCtrl.signup);
router.post('/connexion', userCtrl.login);

module.exports = router;
```

Création des routes utilisateur pour création et authentication.

Authentication (serveur)

```
const User = require('./user');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
```

```
exports.signup = (req, res, next) => {
  bcrypt.hash(req.body.password, 10)
    .then(hash => {
      const user = new User({
        email: req.body.email,
        password: hash
      });
      user.save();
    })
    .then(() => res.status(201).json({ message: 'Utilisateur créé !' }))
    .catch(error => {
      res.status(500).json({ error });
    });
};
```

userController.js

Fonctions associées aux routes de création et d'authentification.

```
exports.login = (req, res, next) => {
  User.findOne({ email: req.body.email })
    .then(user => {
      if (!user) {
        return res.status(401).json({ message: '1 - Paire login/mot de passe incorrecte' });
      }
      bcrypt.compare(req.body.password, user.password)
        .then(valid => {
          if (!valid) {
            return res.status(401).json({ message: '2 - Paire login/mot de passe incorrecte' });
          }
          res.status(200).json({
            userId: user._id,
            token: jwt.sign(
              {userId: user._id},
              'PHRASE_ALEATOIRE_TRES_LONGUE',
              { expiresIn: '24h' }
            )
          });
        })
        .catch(error => res.status(500).json({ error }));
    })
    .catch(error => res.status(500).json({ error }));
};
```

Authentication (serveur)

auth.js

```
const jwt = require('jsonwebtoken');
```

En-tête attendu de la part du frontend.

```
module.exports = (req, res, next) => {
```

```
  try{
```

```
    const token = req.headers.authorization;
```

```
    const decodeToken = jwt.verify(token, 'PHRASE_ALEATOIRE_TRES_LONGUE');
```

```
    const userId = decodeToken.userId;
```

```
    req.auth = {  
      userId: userId  
    };  
    next();
```

On garde le userId pour une utilisation éventuelle
(exemple : un utilisateur ne peut modifier que ses produits.

```
  } catch(error){  
    res.status(401).json([ error ]);  
  }
```

```
};
```

Le middleware suivant
est appelé.

Même phrase que celle utilisée lors de la création du
token dans la fonction d'authentification.

Middleware qui va vérifier le token
envoyé par le client.

Authentication (serveur)

```
const express = require('express');
const router = express.Router();
const controleur = require('./metier');
const auth = require('./auth');

router.get('/', auth, controleur.lireTous);
router.post('/', auth, controleur.creerProduit);
router.get('/:id', auth, controleur.lireProduit);
router.put('/:id', auth, controleur.modifierProduit);
router.delete('/:id', auth, controleur.supprimerProduit);
router.get('/s/:n', auth, controleur.chercheProduit);

module.exports = router;
```

Le middleware d'authentification est ajouté à chacune des routes. L'ordre est important: on vérifie d'abord que l'utilisateur est authentifié avant de lui donner accès.

Authentication (client)

Formulaire d'authentification client.

```
function FormAuth({modToken}){
  const [user, setUser] = useState({"email" : "", "password" : ""});
  const [login, setLogin] = useState(false);

  useEffect(() => {
    if(login){
      fetch('http://localhost:3000/user/connexion',
      {
        method: 'POST' ,
        headers: {
          'Accept': 'application/json',
          'Content-Type': 'application/json'
        },
        body: JSON.stringify(user)
      })
        .then(rep => rep.json())
        .then(rep => {
          localStorage.setItem('jwt-token', rep.token)
          modToken(localStorage.getItem('jwt-token'));
          setLogin(false)
        })
        .catch(err => {console.log(err)});
    }
  }, [login]);

  function handleConnexion(e){
    setLogin(true);
    e.preventDefault();
  }

  return(
    <FormAuth modToken={setToken}/>
  )
}
```

Reçoit le token et utilise localStorage pour le stocker.

modToken permet de faire remonter l'information au composant principal.

Envoie les données d'authentification à la route d'authentification.

```
return (
  <>
    <form method='post'>
      <label htmlFor='text'>Courriel</label>
      <input type='text' id='courriel' value={user.email || ""} onChange={(e) => {
        setUser(previousState => {
          return {...previousState, email: e.target.value}
        }}
      }}/>
      <label htmlFor='pass'>Mot de passe</label>
      <input type='password' id='pass' value={user.password || ""} onChange={(e) => {
        setUser(previousState => {
          return {...previousState, password: e.target.value}
        }}
      }}/>
      <button id='create-product' className='btn btn-primary pull-right m-b-15px create-product-button' onClick={e=>handleConnexion(e)}>
        <span className='glyphicon glyphicon-plus'></span> Connect
      </button>
    </form>
  </>
)
```

Authentication (client)

```
const [token, setToken] = useState(null);
```

```
useEffect(() => {  
  if(read){  
    fetch('http://localhost:3000/product/' + read,  
    {  
      method: 'GET',  
      headers: {  
        'authorization': token  
      }  
    })  
    .then(rep => rep.json())  
    .then( rep => {  
      setProduits(rep);  
    })  
    .catch(err => console.log(err));  
  }  
}
```

Une variable d'état token
contient le token stocké à l'aide
de localStorage.

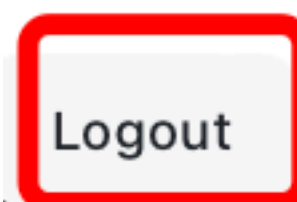
Pour chacune des routes, on envoie
le token dans les en-têtes. Le serveur
attend l'en-tête « authorization ».

Token fait partie des
dépendances de
useEffect donc provoque
un nouvel affichage
lorsqu'il est modifié.

```
}, [read, edit, del, update, addProduct, searchProduct, token]);
```

Authentication (client)

Samsung Galaxy S4	600	3	<button>Read</button> <button>Edit</button> <button>Delete</button>
Vélo	490	2	<button>Read</button> <button>Edit</button> <button>Delete</button>



```
<button onClick={handleLogout}>Logout</button>
```

```
function handleLogout(){  
  localStorage.removeItem('jwt-token');  
  setToken(null);  
}
```

Supprime le token donc l'authentification.

Provoque un nouveau rendu (variable d'état)
qui affichera à nouveau la page mais sans
authentification donc affichage du formulaire
d'authentification.